디지털 논리

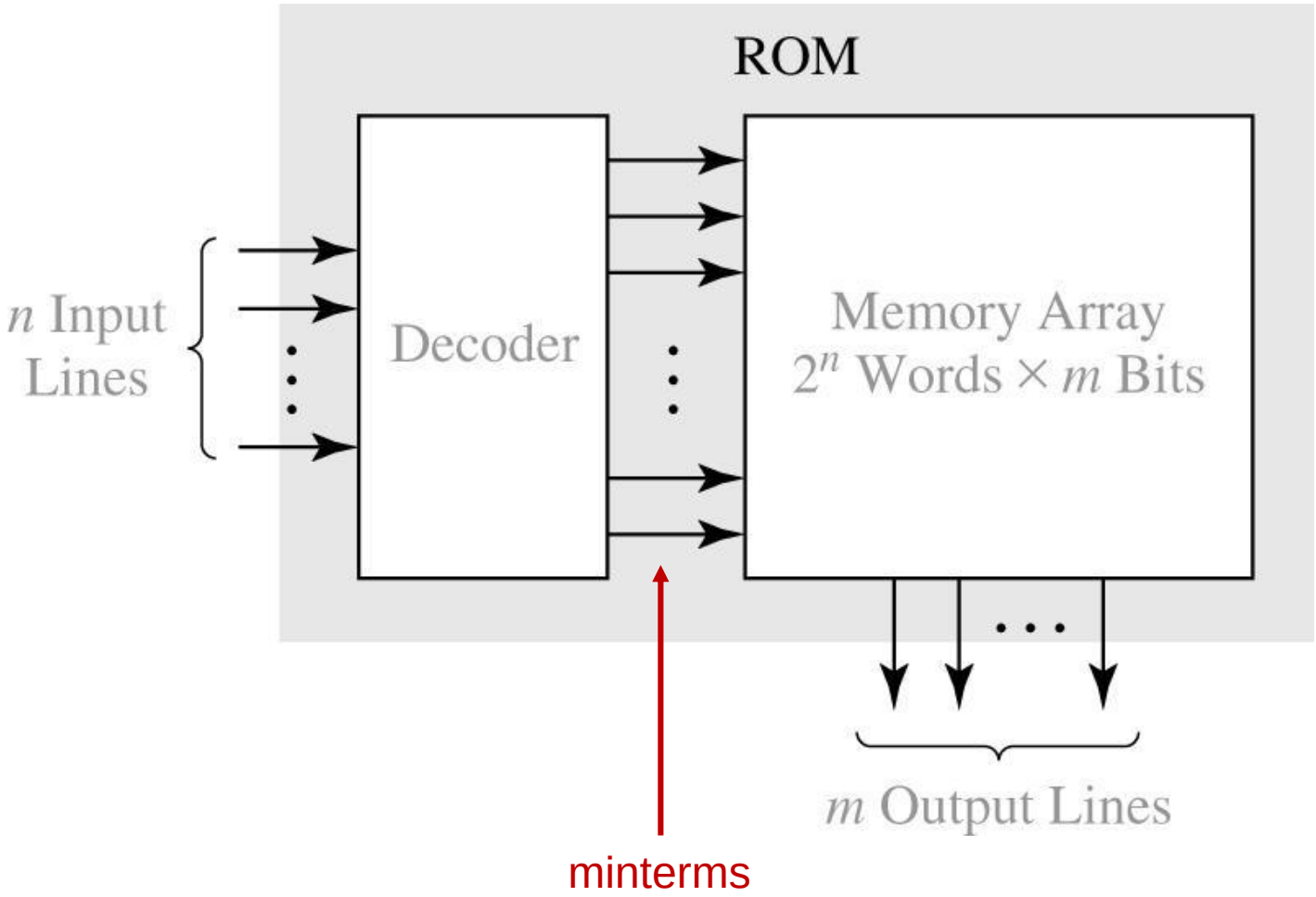
ROM and RAM

이 상 화

ROM (Read Only Memory)

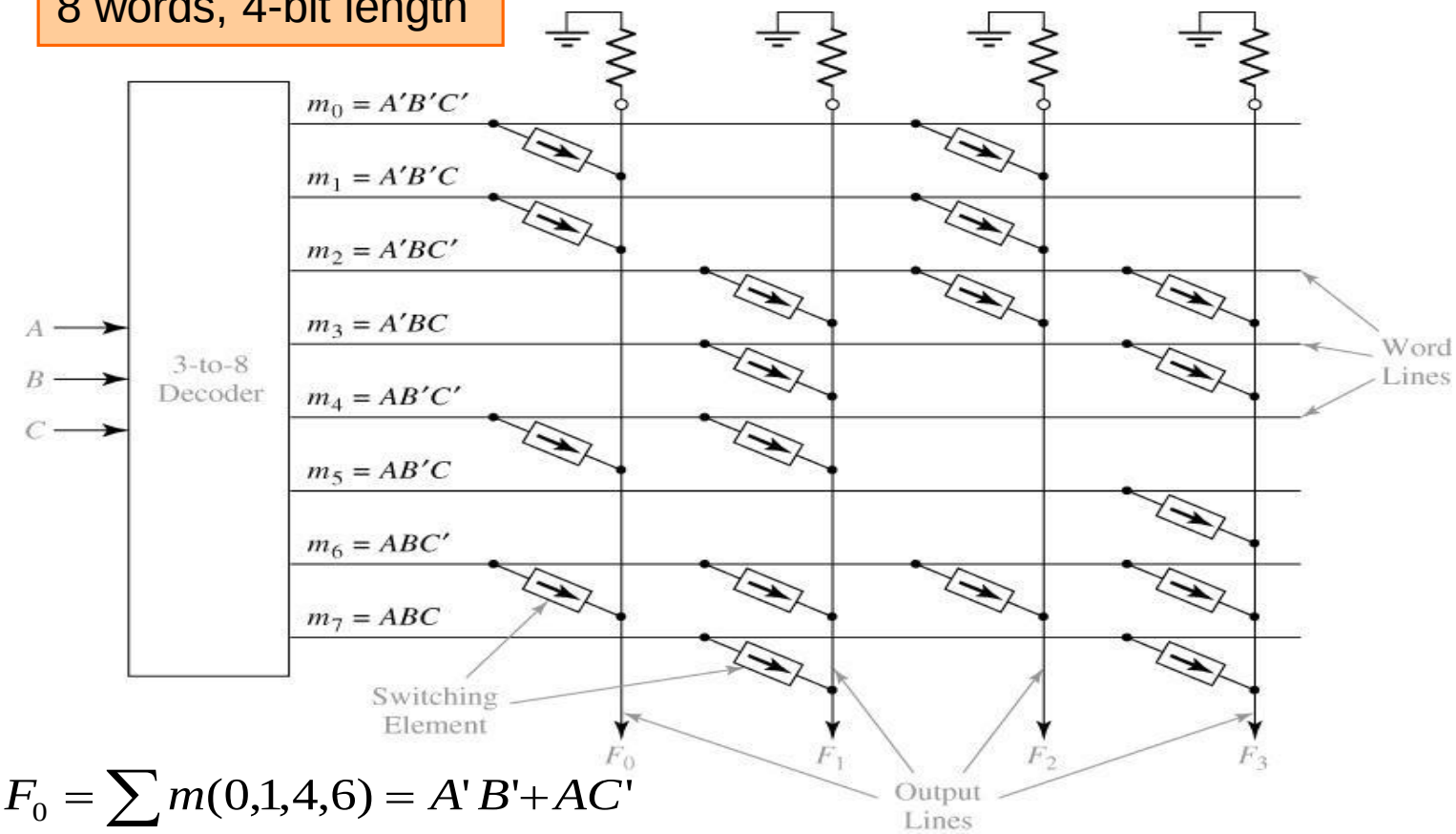
- **Program storage**
  - Boot ROM for personal devices
  - Complete application storage for embedded systems.
- **Actually, a ROM is a combinational circuit, basically a truth-table lookup.**
  - Can perform any combinational logic function by previously stored lookup tables
  - Address inputs = function inputs
  - Data outputs = function outputs

Structure of ROM



Inside of ROM (8x4 ROM)

8 words, 4-bit length



words

1010

1010

0111

0101

1100

0001

1111

0101

$$F_0 = \sum m(0,1,4,6) = A'B' + AC'$$

$$F_1 = \sum m(2,3,4,6,7) = B + AC'$$

$$F_2 = \sum m(0,1,2,6) = A'B' + BC'$$

$$F_3 = \sum m(2,3,5,6,7) = AC + B4$$

Programming codes

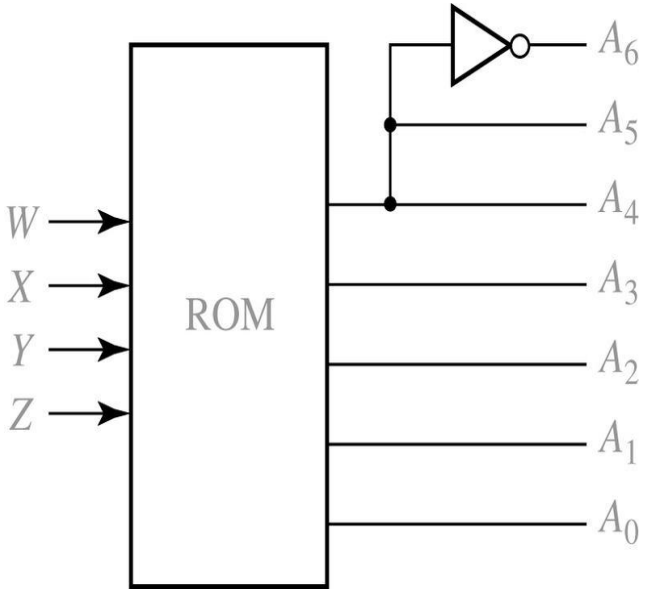
Circuit Design Using ROM (I)

Ex: Hexa-decimal to ASCII Code converter

Input				Hex Digit	ASCII Code for Hex Digit						
W	X	Y	Z		A_6	A_5	A_4	A_3	A_2	A_1	A_0
0	0	0	0	0	0	1	1	0	0	0	0
0	0	0	1	1	0	1	1	0	0	0	1
0	0	1	0	2	0	1	1	0	0	1	0
0	0	1	1	3	0	1	1	0	0	1	1
0	1	0	0	4	0	1	1	0	1	0	0
0	1	0	1	5	0	1	1	0	1	0	1
0	1	1	0	6	0	1	1	0	1	1	0
0	1	1	1	7	0	1	1	0	1	1	1
1	0	0	0	8	0	1	1	1	0	0	0
1	0	0	1	9	0	1	1	1	0	0	1
1	0	1	0	A	1	0	0	0	0	0	1
1	0	1	1	B	1	0	0	0	0	1	0
1	1	0	0	C	1	0	0	0	0	1	1
1	1	0	1	D	1	0	0	0	1	0	0
1	1	1	0	E	1	0	0	0	1	0	1
1	1	1	1	F	1	0	0	0	1	1	0

$$A_5 = A_4$$

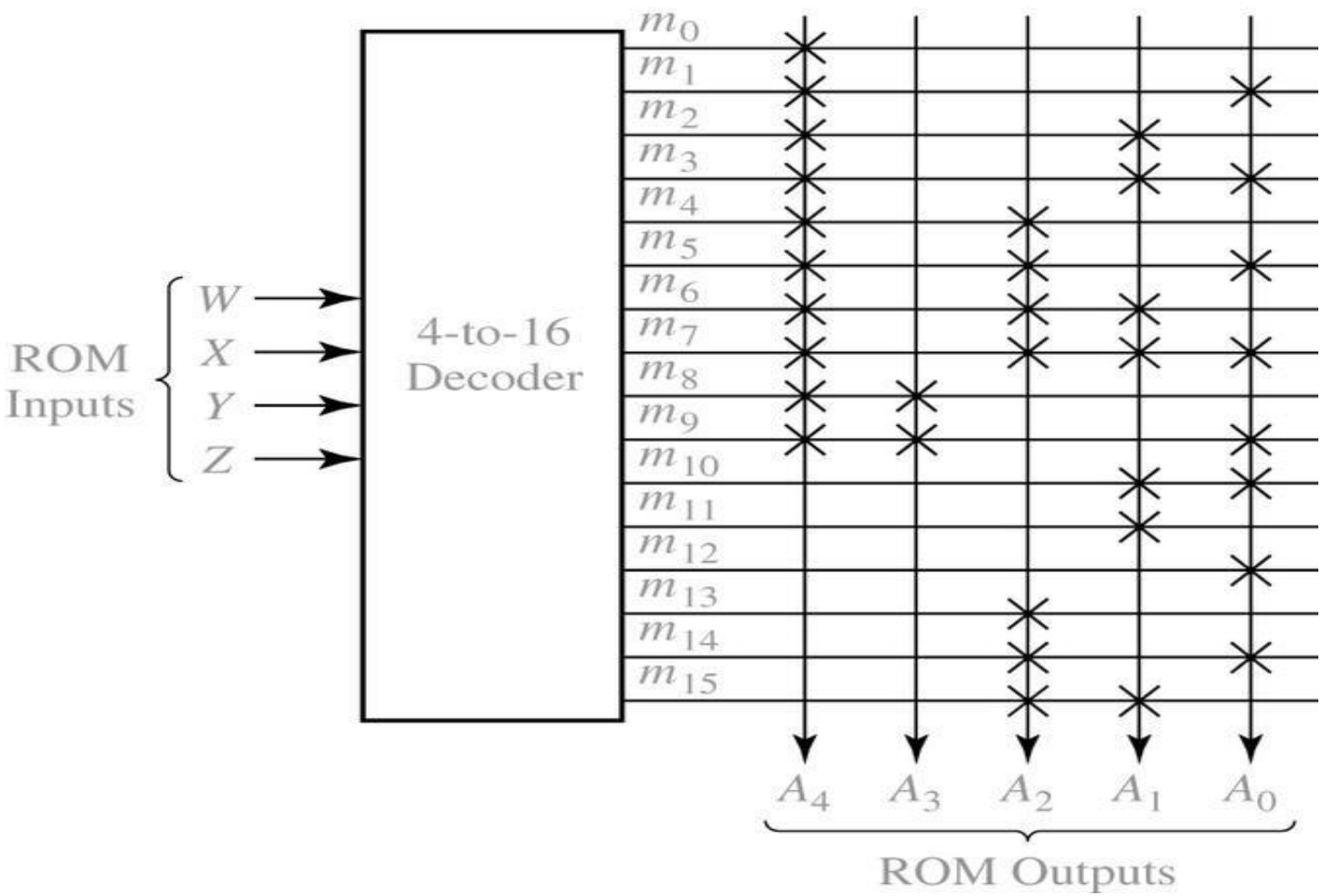
$$A_6 = A_4'$$



Circuit Design Using ROM (II)

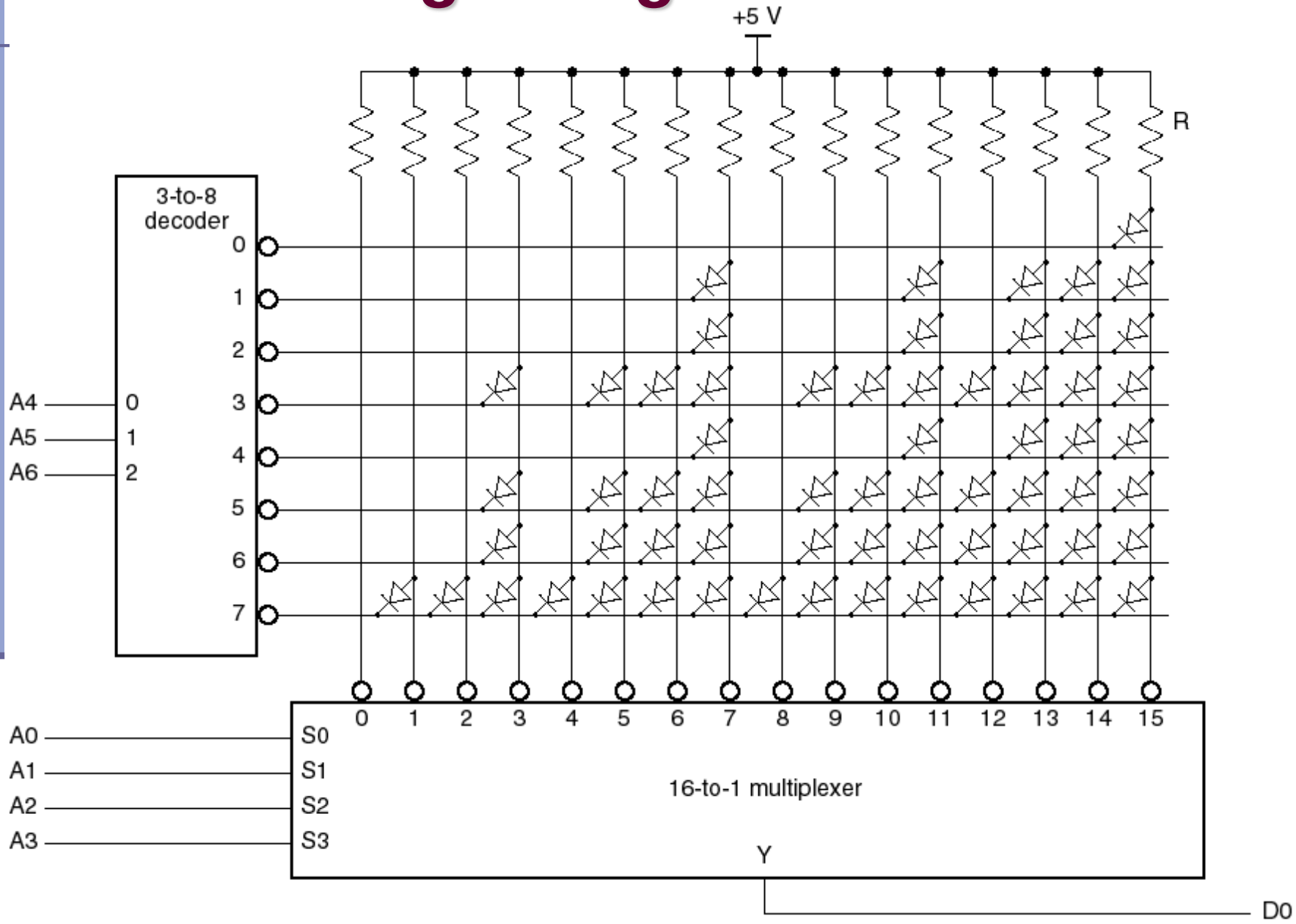
ROM Realization of code converter

X: switching element for “1”

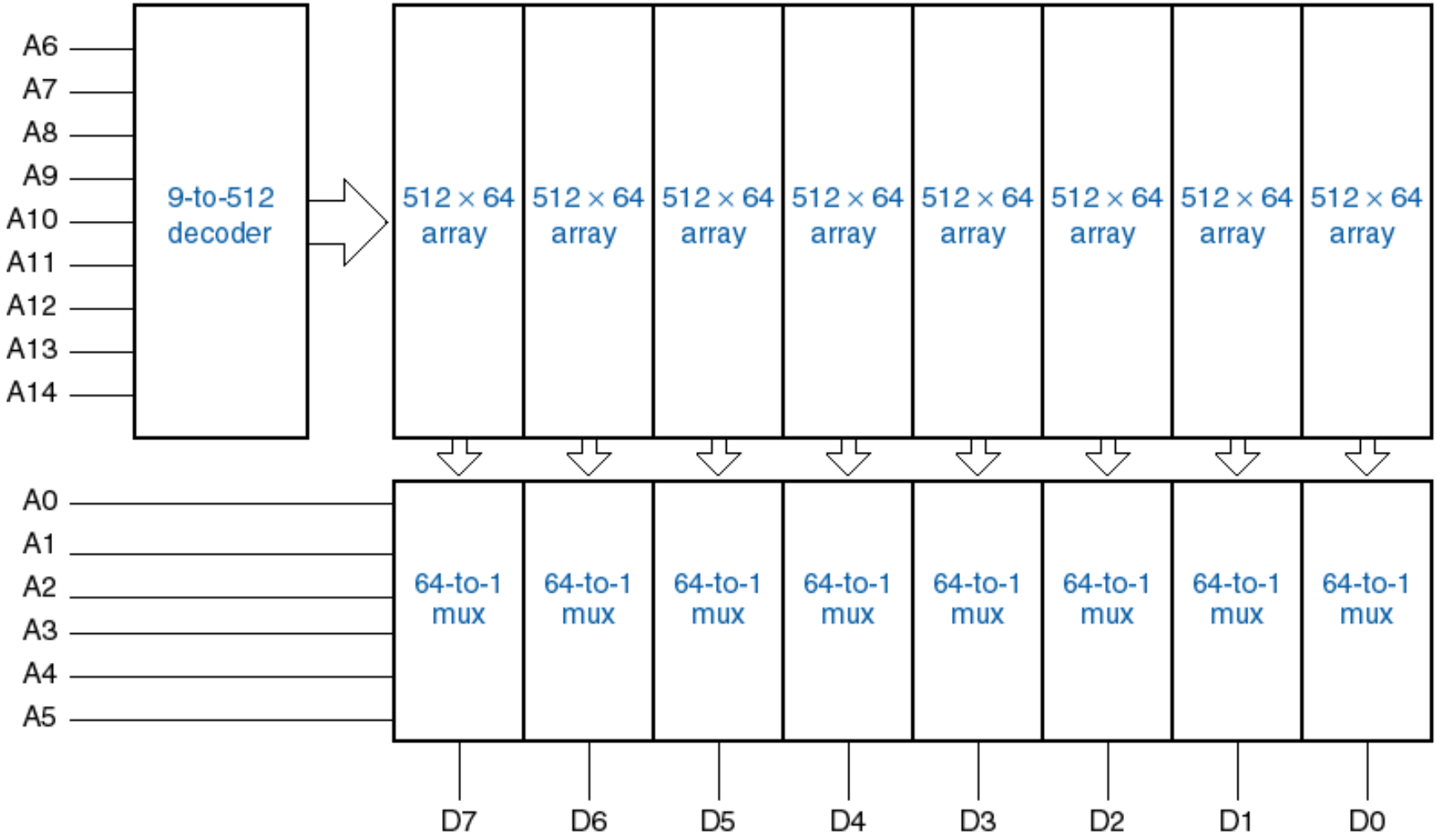


Word ?

2D Decoding Using ROM



Larger Example, 32Kx8 ROM

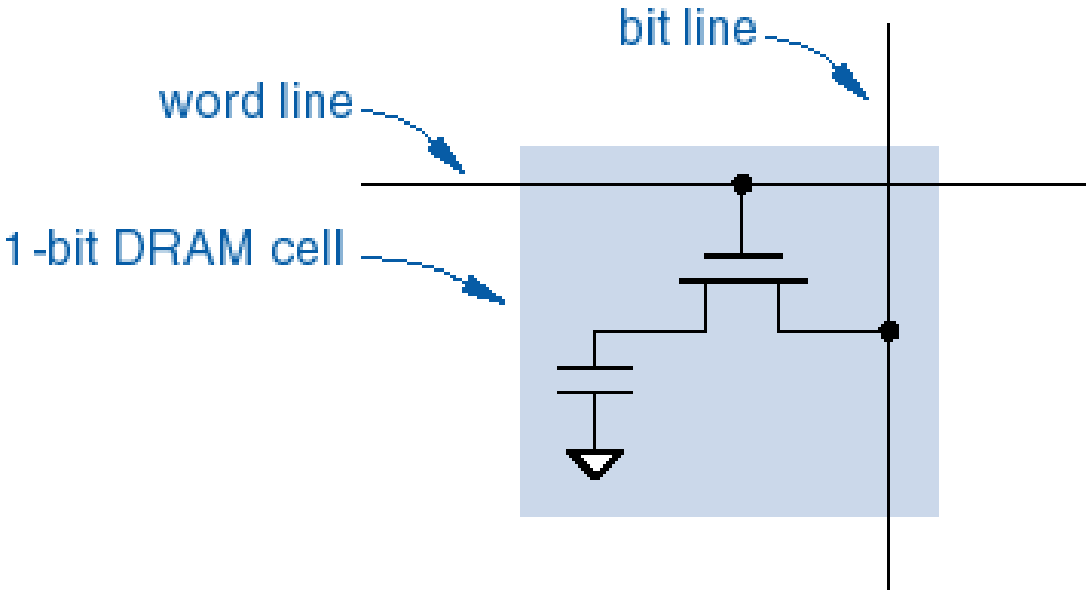


Read/Write Memories

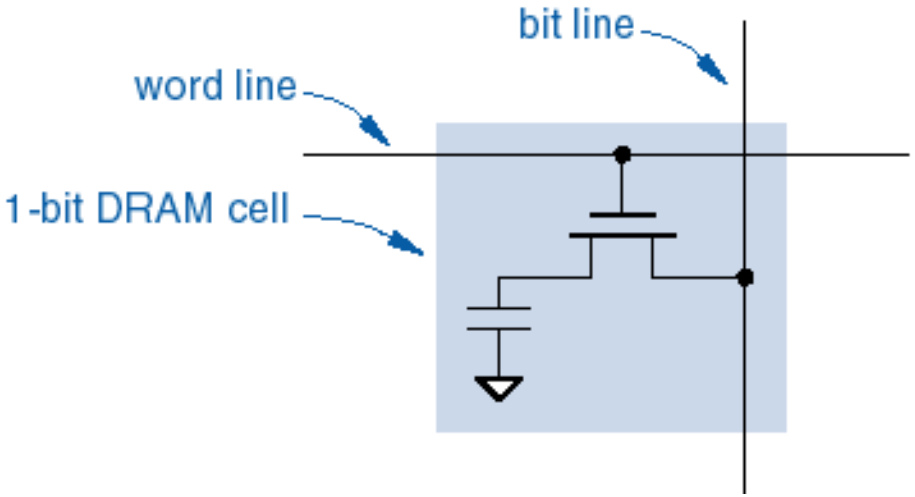
- "RAM" (Random Access Memory)
- Volatility
  - Most RAMs lose their memory when power is removed
  - NVRAM = RAM + battery
- SRAM (Static RAM)
  - Memory behaves like latches or flip-flops
  - Usually 6 transistors
- DRAM (Dynamic Memory)
  - One transistors
  - Memory lasts only for a few milliseconds
  - Must "refresh" locations by reading or writing

DRAM (Dynamic RAM)

- DRAMs use only one transistor per bit
- 1/0 = capacitor charged/discharged

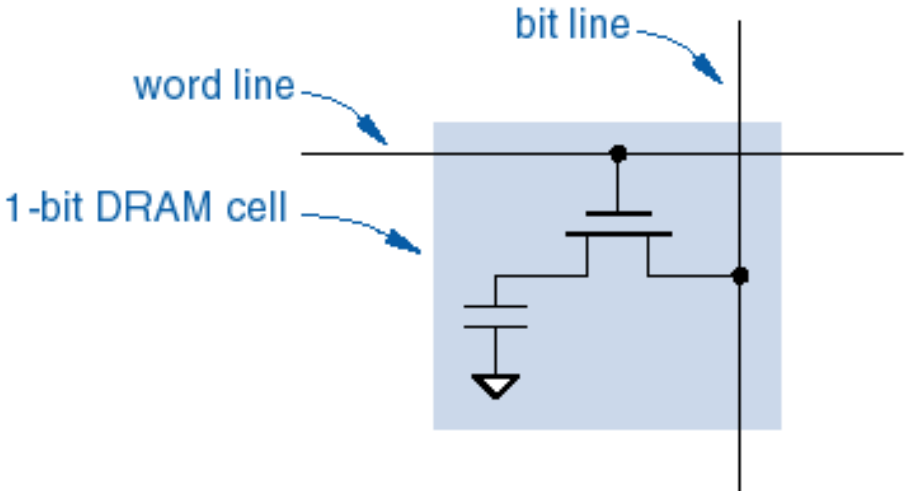


DRAM Read Operations



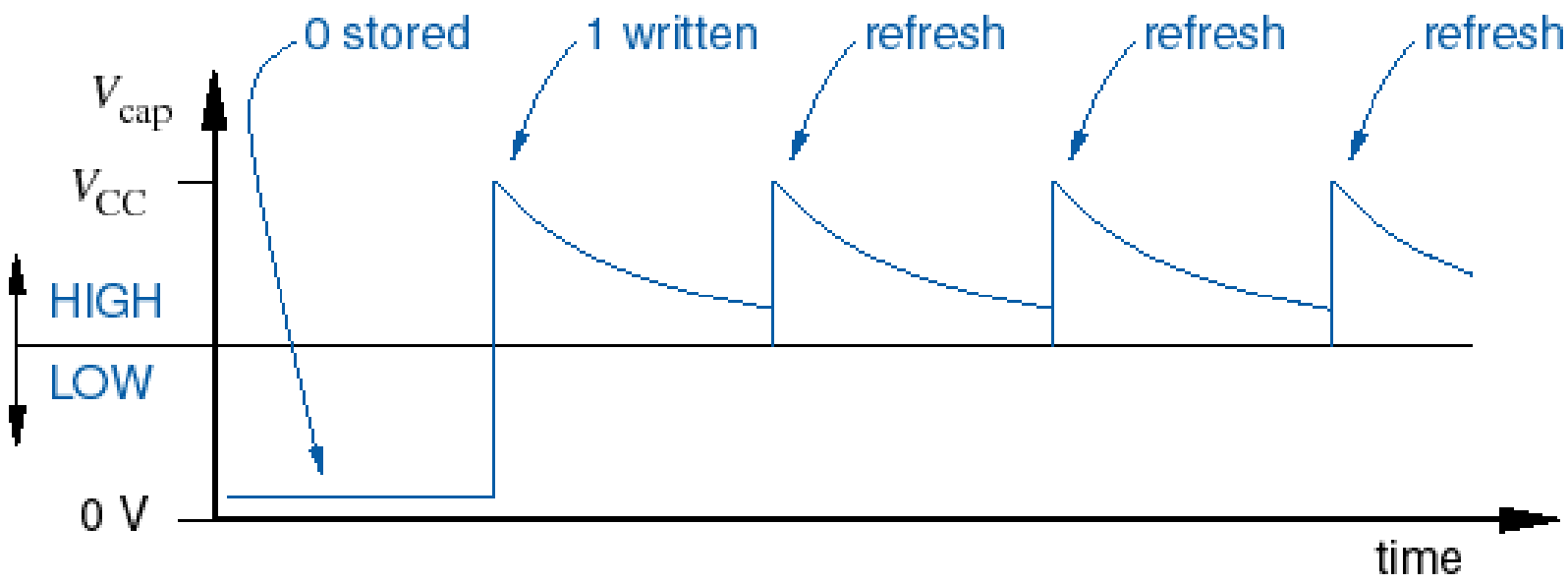
- Precharge bit line to $V_{DD}/2$.
- Take the word line HIGH.
- Detect whether current flows into or out of the cell.
- Note: cell contents are destroyed by the read!
  - Must write the bit value back after reading.

DRAM Write Operations



- Take the word line HIGH.
- Set the bit line LOW or HIGH to store 0 or 1.
- Take the word line LOW.
  - Note: The stored charge for a 1 will eventually leak off.

DRAM charge leakage



- **Typical devices require each cell to be refreshed once every 4 ~ 64 ms.**
- **During "suspended" operation, computers use power mainly for DRAM refresh.**

디지털 논리

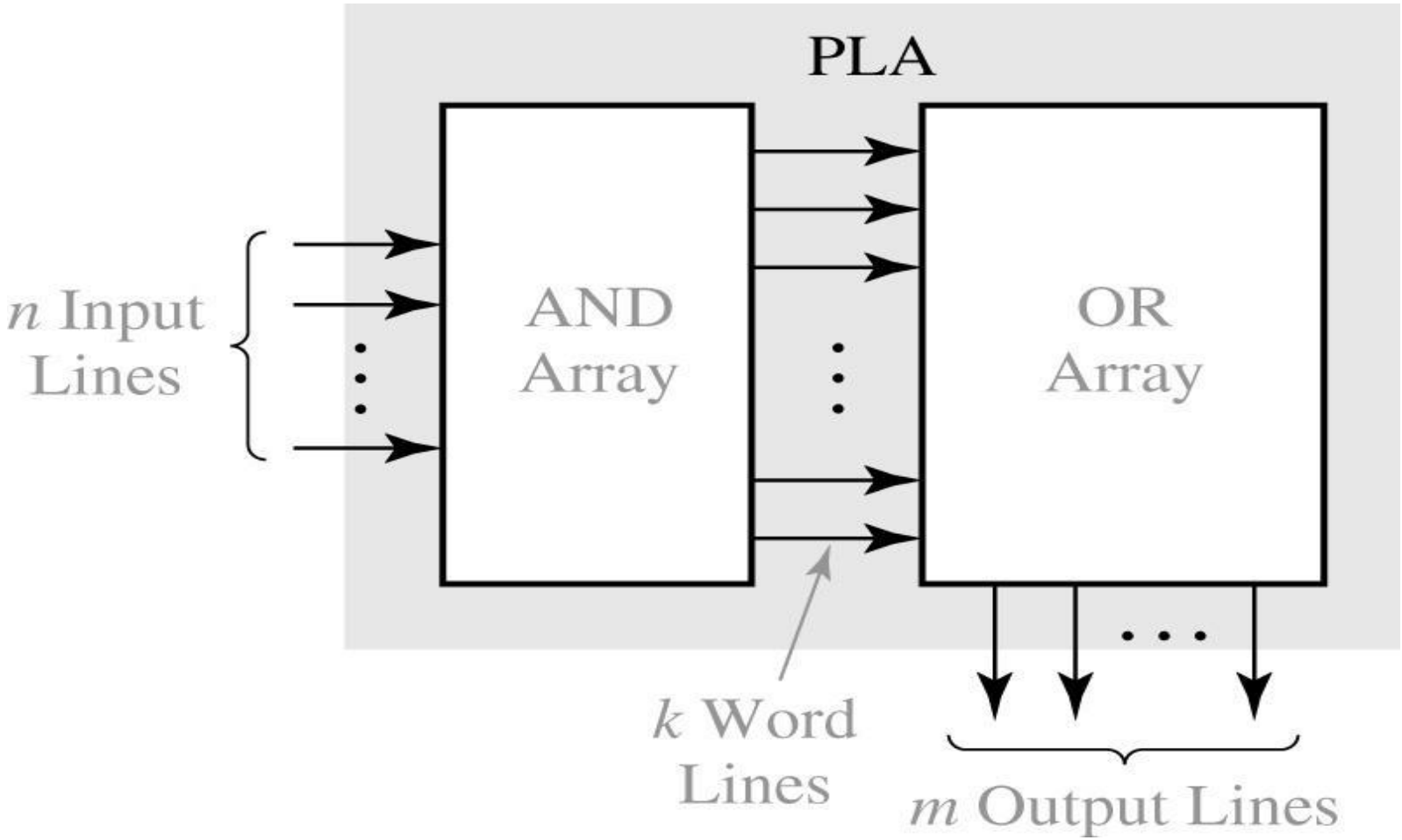
Programmable Logic Array (PLA)

이 상 화

Programmable Logic Array (PLA) (I)

- **Any combinational logic function can be realized as a sum of products.**
- **ROM**
  - Programming minterm expansion
- **PLA**
  - Build a large AND-OR array with lots of inputs and product terms, and programmable connections.
  - *n* inputs
    - AND gates have 2^n inputs -- true and complement of each variable.
  - *m* outputs, driven by large OR gates
    - Each AND gate is programmable to each output's OR gate.

Programmable Logic Array (II)



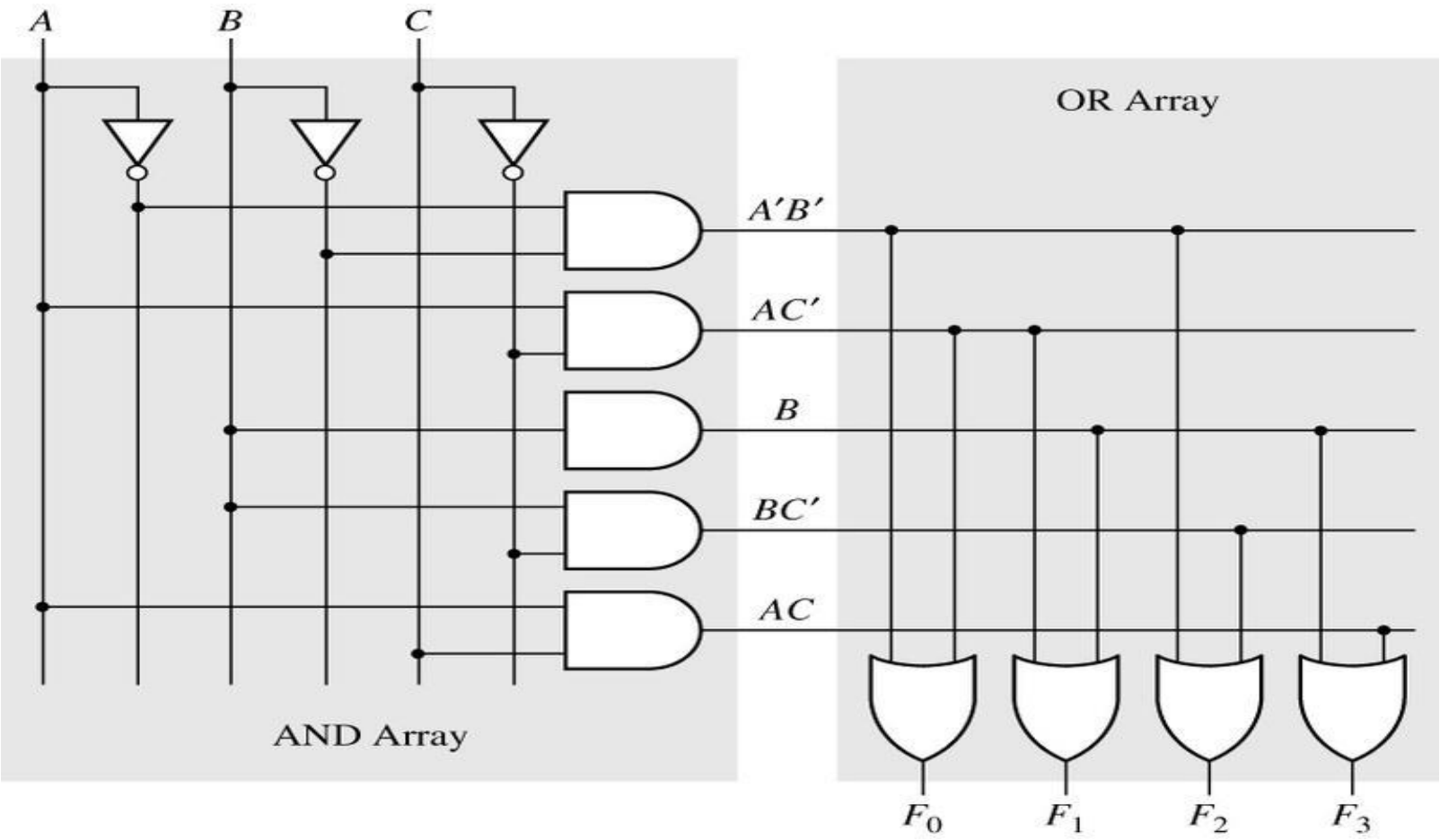
EX: Programmable Logic Array (I)

PLA Table: PLA with 3 inputs, 5 product terms, and 4 outputs

Product Terms	inputs A B C	outputs F0 F1 F2 F3	algebra
A'B'	0 0 -	1 0 1 0	F0=A'B'+AC'
AC'	1 - 0	1 1 0 0	F1=AC'+B
B	- 1 -	0 1 0 1	F2=A'B'+BC'
BC'	- 1 0	0 0 1 0	F3=B+AC
AC	1 - 1	0 0 0 1	

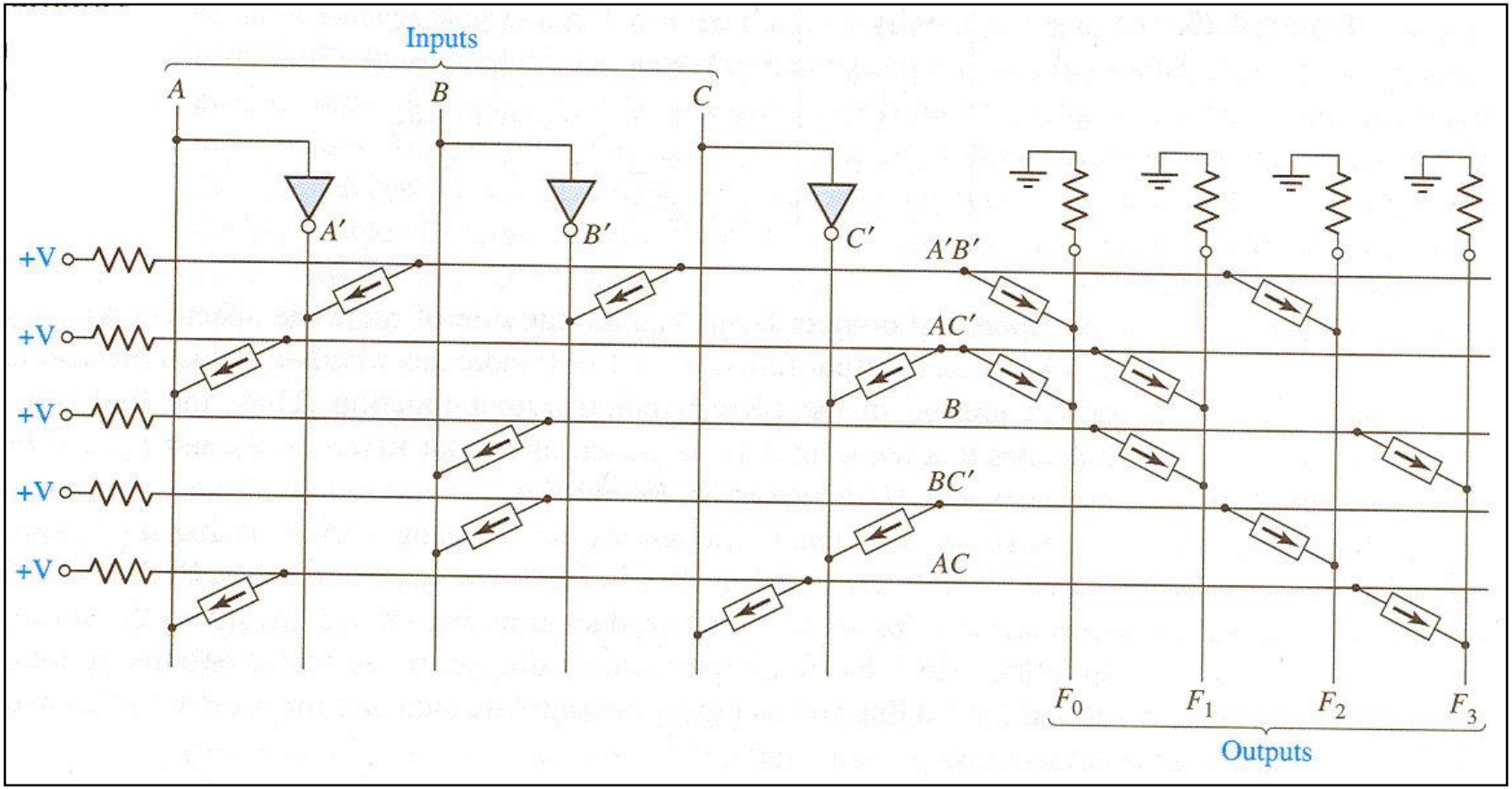
Ex: Programmable Logic Array (II)

AND-OR Array of PLA with 3 inputs, 5 product terms, and 4 outputs

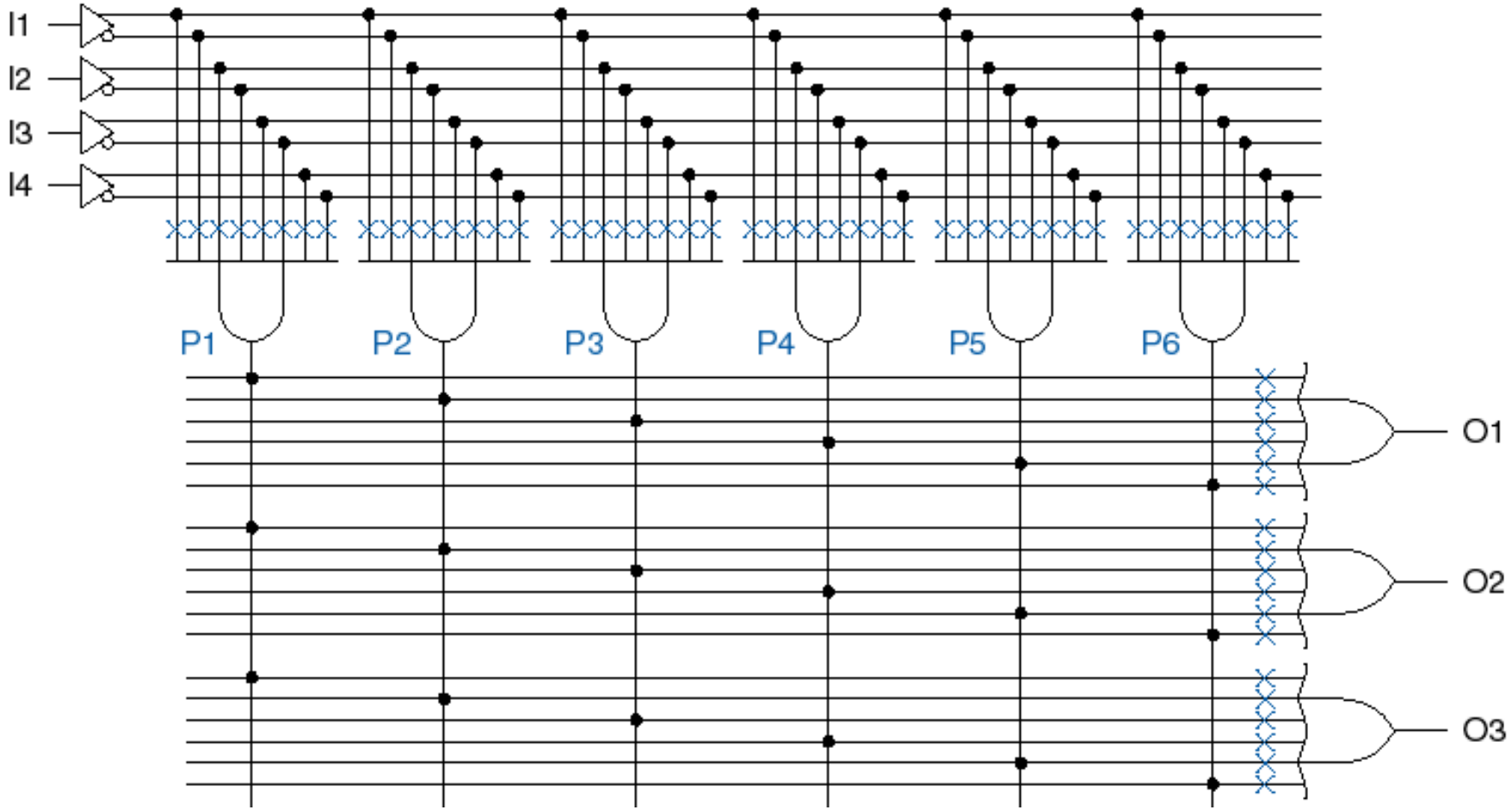


EX: Programmable Logic Array (III)

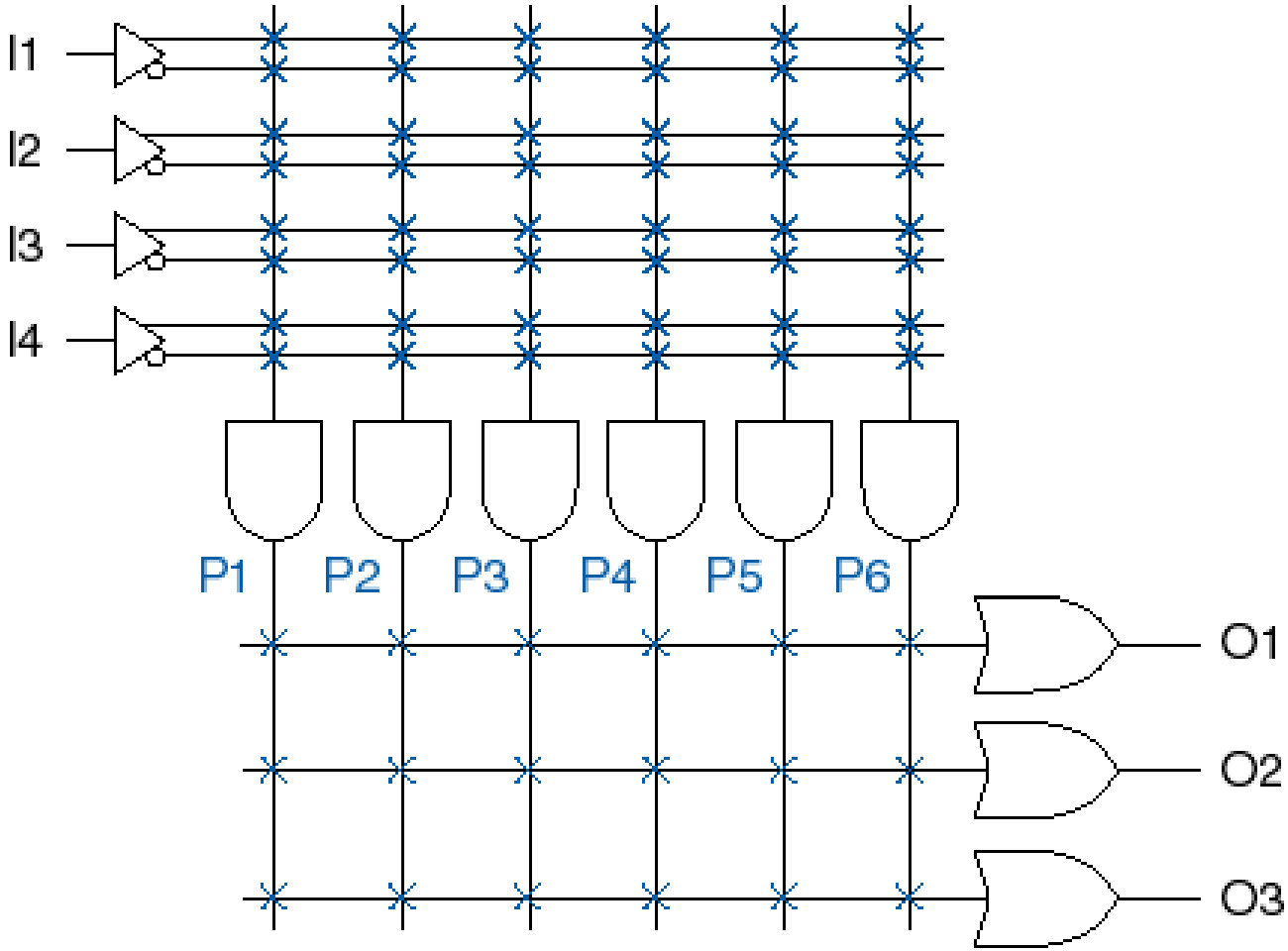
PLA with 3 inputs, 5 product terms, and 4 outputs



Symbol: 4x3 PLA, 6 product terms

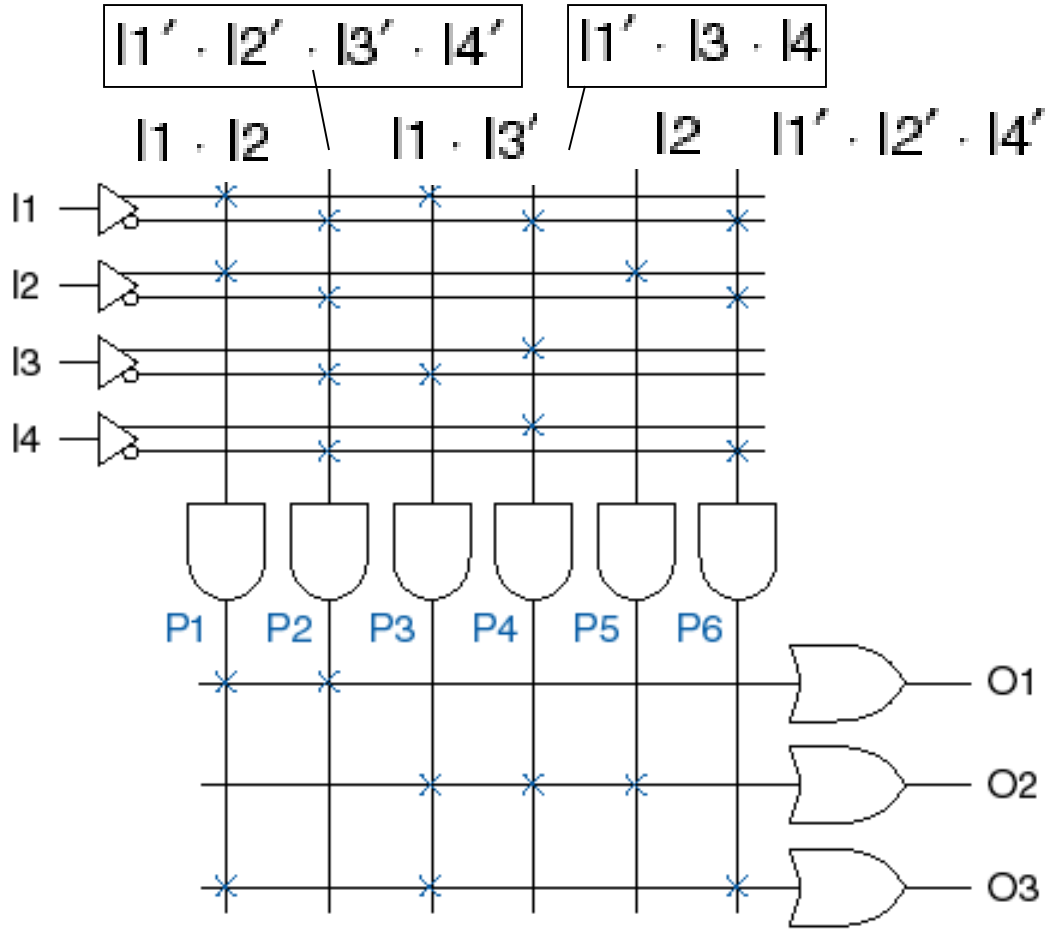


Other Compact Representation



- **Actually, closer to physical layout ("wired logic").**

Ex: PLA Representation



$$O1 = I1 \cdot I2 + I1' \cdot I2' \cdot I3' \cdot I4'$$
$$O2 = I1 \cdot I3' + I1' \cdot I3 \cdot I4 + I2$$
$$O3 = I1 \cdot I2 + I1 \cdot I3' + I1' \cdot I2' \cdot I4'$$

ROM vs PLA

- ROM: minterms expansion
- PLA: general product terms expansion

- ROM: decoder index input (0,1,2,….)
  - Minterm generation
- PLA: logic variables input (A,A’, B,B’,C…)

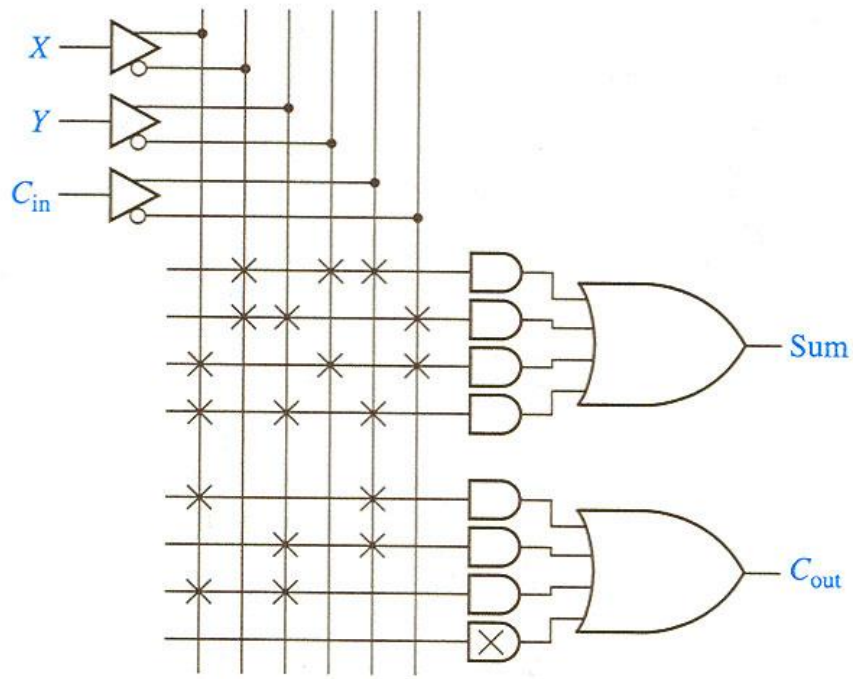
- ROM: small # of variables, large # of products
- PLA: large # of variables, small # of products

Programmable Array Logic (PAL)

- A special case of PLA
- AND gates are programmable, but OR gates are fixed.
- # of OR outputs are limited.
- Ex: Full adder using PAL

$$sum = X'Y'C_{in} + X'YC_{in}^{'} + XY'C_{in}^{'} + XYC_{in}$$

$$C_{out} = YC_{in} + XC_{in} + XY$$



Decomposition of Switching Functions

By Shannon's Theorem

$$f(a,b,c,d) = a'\,f(0,b,c,d) + af(1,b,c,d) = a'\,f_0 + af_1$$

$$\begin{aligned} f(a,b,c,d) &= c'd' + a'b'c + bcd + ac' \\ &= a'(c'd' + b'c + bcd) + a(c'd' + bcd + c') \\ &= a'(c'd' + b'c + cd) + a(c' + bd) = a'\,f_0 + af_1 \end{aligned}$$

Ex: 5 variables

$$f(a,b,c,d,e) = a'\,f(0,b,c,d,e) + af(1,b,c,d,e) = a'\,f_0 + af_1$$

Ex: 6 variables

$$G(a,b,c,d,e,f) = a'b'G_{00} + a'bG_{01} + ab'G_{10} + abG_{11}$$

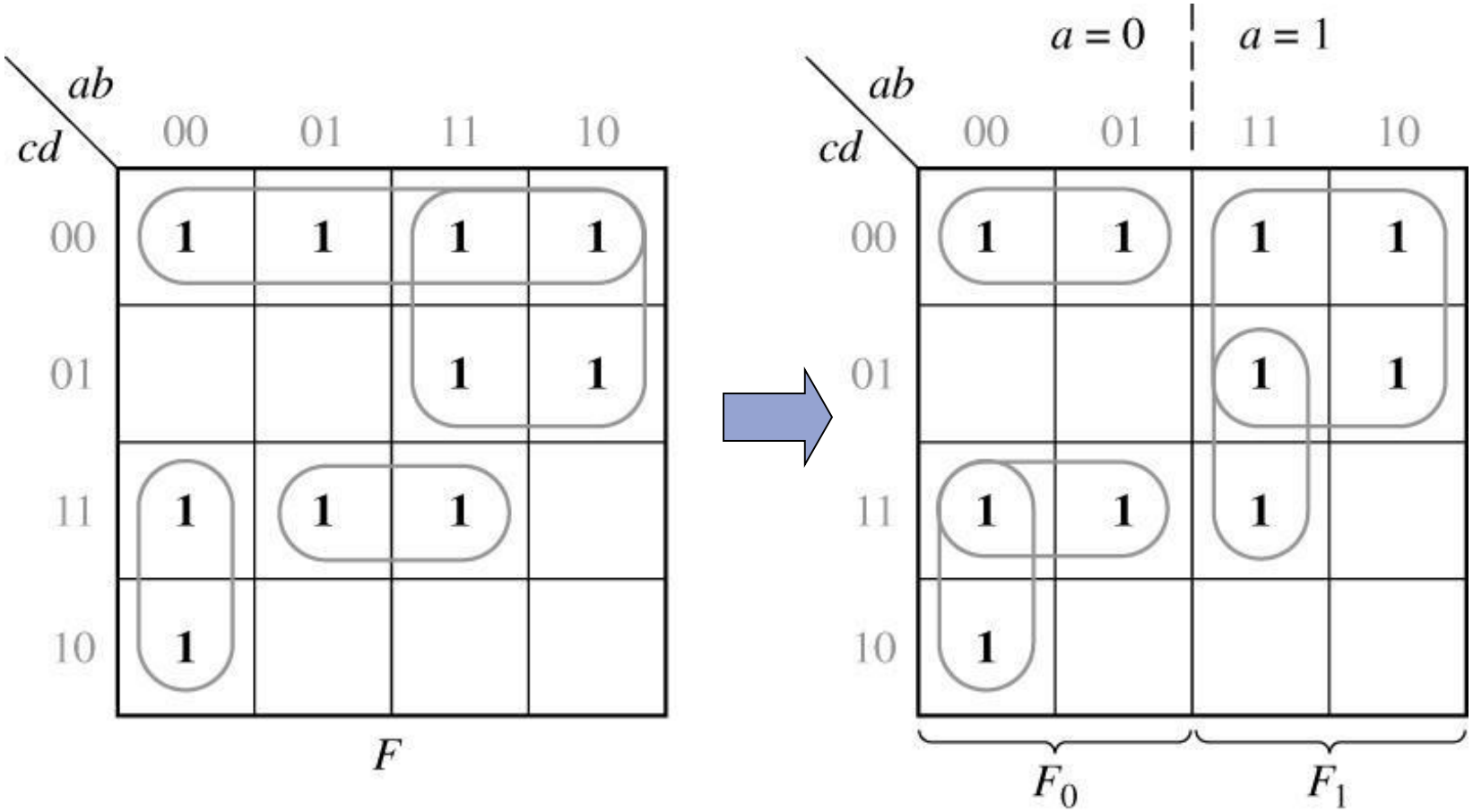
$$G(a,b,c,d,e,f) = a'G(0,b,c,d,e,f) + aG(1,b,c,d,e,f) = a'G_0 + aG_1$$

$$G_0 = b'G(0,0,c,d,e,f) + bG(0,1,c,d,e,f) = b'G_{00} + bG_{01}$$

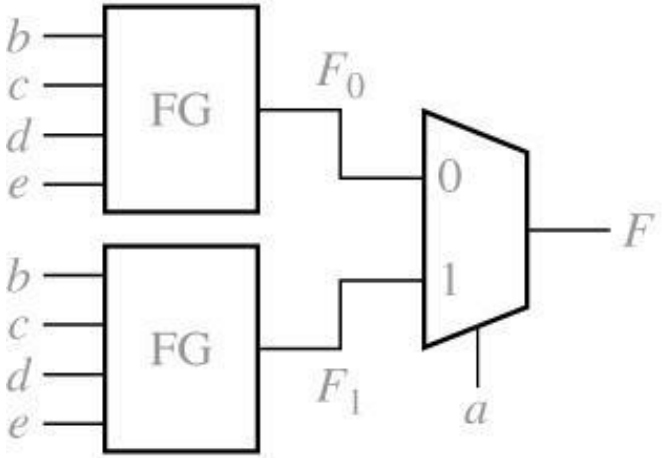
$$G_1 = b'G(1,0,c,d,e,f) + bG(1,1,c,d,e,f) = b'G_{10} + bG_{11}$$

Function Decomposition

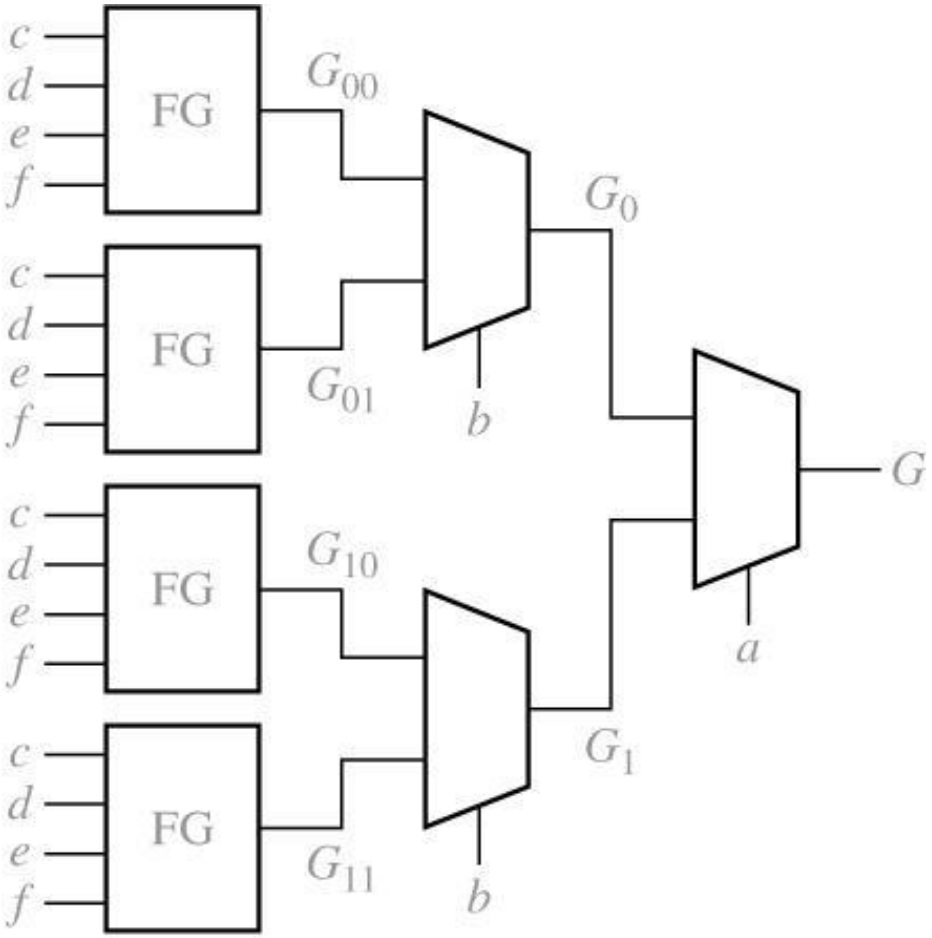
Using Shannon's decomposition theorem to reduce the number of variables to 4



Implementation of Many Variable-function



(a) 5-variable function



(b) 6-variable function